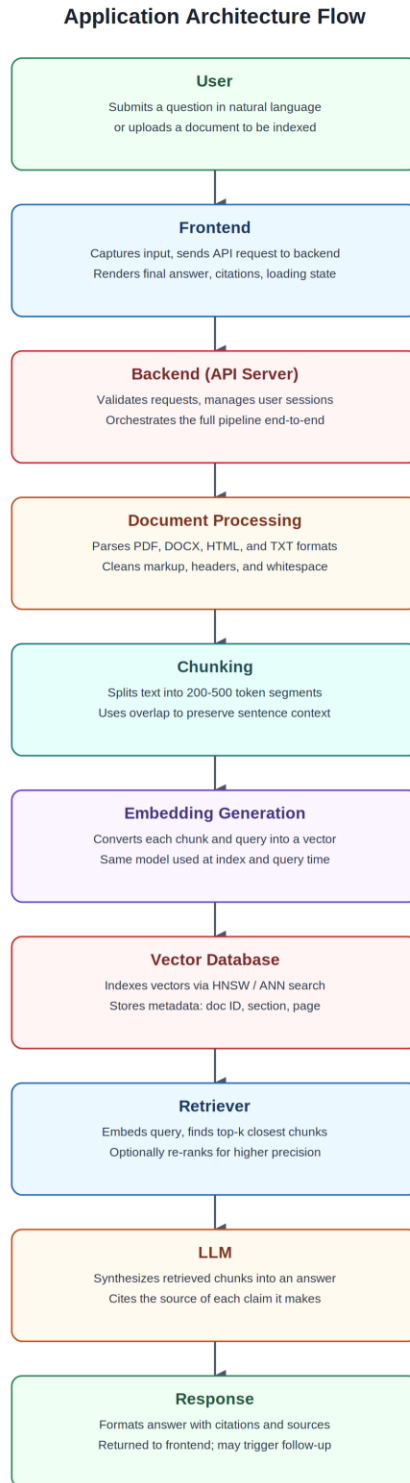


System Architecture Documentation

This document describes the end-to-end architecture of the application, tracing a single request from user input to final response. The system follows a linear pipeline in which each component transforms its input and passes the result to the next stage. Figure 1 shows the complete flow; each component is explained below in the order data moves through the system.



Steps 4-7 run once during ingestion; steps 1-3 and 8-10 run on every query

1. User

The person interacting with the system. The user initiates the workflow by asking a question in natural language or uploading a document to be indexed.

- **Role:** originates every request; the entire pipeline exists to serve this input.
- **Input types:** typed queries, file uploads, or both in document-QA workflows.

2. Frontend

The client-facing interface (web or mobile UI) through which the user submits input and views results.

- **Responsibilities:** captures user input, performs basic client-side validation, sends requests to the backend API, and renders the final response and any citations.
- **Design note:** should show loading/streaming states, since generation can take several seconds.

3. Backend

The API server that receives requests from the frontend and orchestrates every downstream component in the correct order.

- **Responsibilities:** authenticates requests, validates payloads, manages user sessions/history, and coordinates calls to processing, retrieval, and generation services.
- **Design note:** acts as the single entry point, so all business logic and error handling is centralized here rather than in the frontend.

4. Document Processing

Prepares raw uploaded files for indexing by extracting clean, usable text.

- **Responsibilities:** parses formats such as PDF, DOCX, and HTML, strips irrelevant markup/headers/footers, and normalizes encoding and whitespace.
- **Output:** a clean plain-text version of each document, ready for splitting.

5. Chunking

Splits long documents into smaller, indexable segments, since embedding models and LLMs both have limited input capacity.

- **Responsibilities:** divides text into overlapping chunks (commonly 200-500 tokens) along sentence or paragraph boundaries where possible.
- **Why it matters:** chunk quality directly determines what the retriever can later find; poor chunking is the most common cause of weak answers.

6. Embedding Generation

Converts each text chunk, and later each user query, into a dense numerical vector that captures its meaning.

- **Responsibilities:** runs each chunk through an embedding model so that semantically similar text ends up close together in vector space.
- **Design note:** the same embedding model must be used at ingestion time and at query time for the vectors to be comparable.

7. Vector Database

Persistently stores the generated vectors together with their source text and metadata, and serves fast similarity search.

- **Responsibilities:** indexes vectors (e.g., via HNSW/ANN), stores metadata such as document ID and section, and supports filtering during search.
- **Design note:** this is the system's long-term memory; it is populated once during ingestion and read from on every query.

8. Retriever

At query time, searches the vector database for the chunks most relevant to the user's question.

- **Responsibilities:** embeds the incoming query, performs a similarity search for the top-k closest chunks, and optionally re-ranks them for precision.
- **Output:** a short list of the most relevant text passages to hand to the LLM.

9. LLM

The large language model that reads the user's question together with the retrieved chunks and produces a natural-language answer.

- **Responsibilities:** synthesizes the retrieved context into a coherent, grounded answer and, where configured, cites the source of each claim.
- **Design note:** instructions in the prompt constrain the model to rely on the provided context rather than its own unverified memory.

10. Response

The final output delivered back to the user through the frontend.

- **Responsibilities:** formats the answer (text, citations, follow-up suggestions) and returns it to the frontend for display.
- **End of cycle:** the user may then ask a follow-up question, restarting the flow from step 1.

Summary: Documents flow once through Document Processing, Chunking, Embedding Generation, and the Vector Database during ingestion. Every user query then flows through Frontend, Backend, Retriever, and LLM to produce a Response, closing the loop back to the User.